

COS341 Group Project (Group) - SPL Compiler Front-End

A lexical analyzer, LL(1) top-down parser, syntax tree visualizer, and semantic analyzer for the Simple Programming Language (SPL), built for COS341.

Members

Member Name	Student Number	Role
Christopher Adolph	23535548	Speaker
Dewald Colesky	23536030	Developer
Heinrich Romer	23538181	Developer
Jonty Honey	23536862	Developer

What this project does

- Phase 1: Lexing, parsing, and syntax tree generation (writes `tree.xml`)
- Phase 2a: Semantic analysis, including scope resolution, symbol table construction, and name/function rule checking

Requirements

Python 3.10 or newer.

For the optional graph visualizer only, you also need Graphviz.

To create an executable PyInstaller is required

Build the executable

Install PyInstaller:

```
pip install pyinstaller
```

Build a single-file executable from `parser.py` . (PyInstaller does not cross compile, a build made on Linux only runs on Linux, a build made on Windows only runs on Windows):

```
pyinstaller --onefile --name group-8 main.py
```

Usage

Executable Usage

Run the executable (program file must be next to executable)

```
./group-8 <filename>
```

Developer Usage

Run the parser and semantic analyzer manually (must be next to main.py):

```
python3 main.py <filename>
```

Generate a visual graph of the tree (optional, requires Graphviz):

```
python3 graph_gen.py
```

This converts `tree.xml` into `syntax_tree.png`.

On success, this writes `tree.xml` (the syntax tree in the required ID, CONTENTS, CHILDREN, PARENT format).

On failure, it prints one of:

- `SYNTAX ERROR: ...` if the input does not match the SPL grammar
- `LEXICAL ERROR: ...` if the input contains an invalid token
- One or more `SEMANTIC ERROR: ...` lines if the input is syntactically valid but violates a naming, scope, or function rule

Important note about the end of input

The pseudo symbol `$` is NOT literally included at the end of a real test file. `$` is only used to talk about the grammar and the parser, it is not an actual member symbol of the SPL language itself. Our lexer automatically produces an end of file `$` token once it reaches the end of the input, whether or not the file itself contains a literal `$` character.

Project structure

File	Owner	Purpose
<code>tokens.py</code>	shared	Shared <code>Token</code> class, keyword and symbol tables
<code>lexer.py</code>	shared	Tokenizes SPL source into a stream of <code>Token</code> s
<code>parser.py</code>	shared	Recursive descent LL(1) parser, builds the syntax tree and writes <code>tree.xml</code>
<code>graph_gen.py</code>	shared	Renders <code>tree.xml</code> as a <code>syntax_tree.png</code> image using Graphviz
<code>tree_crawl.py</code>	Colesky	Walks the syntax tree, assigns scope levels, builds the scope hierarchy
<code>symbol_table.py</code>	Jonty	Builds the symbol table from the scope tree, assigns unique internal (<code>sys###</code>) names
<code>variable_rules.py</code>	Heinrich	Checks variable declaration and usage rules: ancestor lookup, no duplicates, no parameter masking
<code>function_rules.py</code>	Christopher	Checks function rules: name uniqueness, strictly hierarchic call resolution, recursion detection
<code>semantic_checker.py</code>	shared	Combines all semantic rule checks into one entry point

Grammar

```
SPL_PROG -> P $
P        -> V_DECL : F_DECL : ALGO
V_DECL   -> epsilon | USER-DEFINED-NAME V_DECL
F_DECL   -> epsilon | F_TYPE F_DECL
F_TYPE    -> void NAME ( V_DECL ) { P return }
          | num  NAME ( V_DECL ) { P return ( TERM ) }
ALGO     -> epsilon | INSTR ; ALGO
INSTR    -> USER-DEFINED-NAME INSTRTAIL | print OUTP | nop
          | comment STRING | BRANCH | LOOP
INSTRTAIL -> ( INPUT ) | = TERM
TERM     -> USER-DEFINED-NAME TERMTAIL | NUM
          | mod/add/sub/mul/div ( TERM TERM ) | neg ( TERM )
BRANCH   -> if BOOL then { ALGO } else { ALGO }
BOOL     -> not(BOOL) | and/or(BOOL BOOL) | eq/larger/lesser(TERM TERM)
LOOP     -> COND BOOL do { ALGO } | do { ALGO } COND BOOL
COND     -> while | until
```

Left factored for LL(1) parsing. `INSTRTAIL` and `TERMTAIL` resolve the shared `USER-DEFINED-NAME` prefix with one token of lookahead.

Semantic rules implemented

Variables: a declared name must exist in the usage's own scope or an ancestor scope, with the nearest declaration winning. No two declarations of the same kind are allowed at one scope level. A function's local variables may not mask its own parameters.

Functions: no duplicate function names are allowed at a scope level. Function calls are strictly hierarchic, meaning a call must resolve within the caller's own immediate scope, never an ancestor's. This one rule structurally blocks both direct and indirect recursion, along with sideways calls between sibling functions' private helpers.

Test cases

Basic statements

Assignment:

```
: : #x = 5 ; $
```

Arithmetic addition:

```
: : #x = add ( 3 4 ) ; $
```

Negation:

```
: : #x = neg ( 9 ) ; $
```

Function call instruction:

```
: : #log ( 1 ) ; $
```

Branching (if then else):

```
: : if eq ( #x 0 ) then { print ( #x ) ; } else { nop ; } ; $
```

While loop:

```
: : while larger ( #x 0 ) do { #x = sub ( #x 1 ) ; } ; $
```

Do until loop:

```
: : do { nop ; } until eq ( #x 0 ) ; $
```

Advanced benchmarks

Recursion and arithmetic (factorial). Tests num function declarations, local variable declarations, recursive calls, and nested arithmetic expressions:

```
#result #temp :
num #fact ( #n ) {
  #res : :
  if eq ( #n 0 ) then {
    #res = 1 ;
  } else {
    #res = mul ( #n #fact ( sub ( #n 1 ) ) ) ;
  } ;
  return ( #res )
} :
#result = #fact ( 5 ) ;
print ( #result ) ;
$
```

Control flow, strings, and void functions. Tests void functions, nested loops (while and do until), multi variable conditions (and, or, not), and string output:

```
#i #j #max #found :
void #reset ( #v ) {
  : :
  #v = 0 ;
  nop ;
  return
} :
#i = 0 ;
#max = 10 ;
while lesser ( #i #max ) do {
  #j = 0 ;
  do {
    if and ( not ( eq ( #i #j ) ) larger ( mod ( #i 2 ) 0 ) ) then {
      print "match found" ;
      #found = 1 ;
    } else {
      nop ;
    } ;
    #j = add ( #j 1 ) ;
  } until or ( eq ( #j #max ) eq ( #found 1 ) ) ;
  #i = add ( #i 1 ) ;
} ;
#reset ( #i ) ;
$
```

Nested scopes and prefix expression trees. Tests function declarations inside local scopes and deeply nested operator trees:

```
#val :
num #outer ( ) {
  :
  num #inner ( #a #b ) {
    : :
    return ( add ( #a #b ) )
  } :
  #val = #inner ( neg ( 10 ) 20 ) ;
  return ( #val )
} :
#val = #outer ( ) ;
print ( add ( sub ( mul ( div ( neg ( 100 ) 2 ) 3 ) 4 ) mod ( 10 3 ) ) ) ) ;
$
```

Semantic rule tests live in `tests/` :

File	Tests
<code>NESTED.txt</code> / <code>SERIALNESTED.txt</code>	Valid deeply nested and sibling function declarations, 0 errors expected
<code>MaskTest.txt</code>	Parameter masked by a local variable
<code>tests/VariableErrors.txt</code>	Duplicate declaration, masking, and undeclared variable, combined
<code>tests/VariableValid.txt</code>	Valid variable usage across nested scopes
<code>tests/DirectRecursion.txt</code>	A function calling itself
<code>tests/IndirectRecursion.txt</code>	Two functions calling each other
<code>tests/SidewaysCall.txt</code>	Sibling functions attempting to call each other's private sub-functions
<code>tests/DupFunc.txt</code>	Two functions with the same name at one level

Status

- ☒ Phase 1: Lexer, Parser, Syntax Tree (`tree.xml`)
- ☒ Phase 2a: Scope analysis, Symbol Table, Variable and Function semantic rules
- ☒ Executable build and packaging for Phase 1 submission
- ☐ Phase 2b / later phases (TBD)